Image Processing

resize, convert, cache

Watch the 4-min video

In one line: never serve the original upload: create smaller variants in modern formats (WebP, AVIF, with JPEG fallback), either pre-generated by an asynchronous worker or transformed on demand and cached at the CDN, with guard rails on sizes, pixel counts and decoders.

1 An 8 MB thumbnail

A user uploads a 12-megapixel phone photo (4032 × 3024, about 8 MB). The product page shows it at 300 × 225 CSS pixels. Serve the original and every visitor downloads 8 MB for an image that needs roughly 30–60 KB, pages load slowly, Largest Contentful Paint suffers, and mobile users pay for the data. Phones also upload formats browsers can't all show (HEIC from iPhones), rotated via EXIF tags, in wide-gamut colour, with GPS coordinates embedded. Images need processing before they're served.

2 Asynchronous resizing after upload

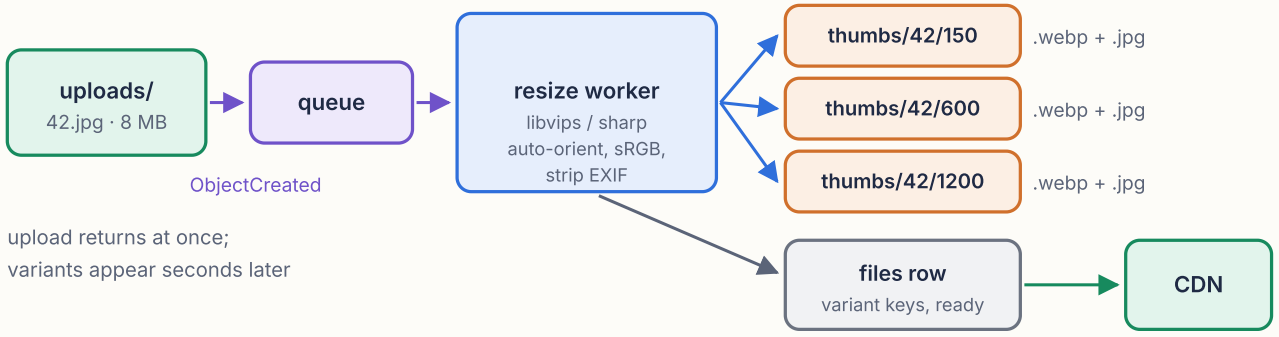


Figure 1. Pre-generating: an upload event triggers a worker that writes every variant the app needs under predictable keys.

The upload goes straight to the bucket (lesson 4); an `ObjectCreated` event puts a job on a queue; a worker downloads the original, creates each size and format, writes them as separate objects and records them in the database. The upload request returns immediately and the UI shows a placeholder until the row is `ready`. Processing steps, in order:

1. **Decode safely**: check the declared dimensions before decoding (see section 5).
2. **Auto-orient**: apply the EXIF Orientation tag to the pixels, *then* strip metadata, or photos appear sideways.
3. **Colour**: convert to sRGB (or keep the ICC profile); dropping a Display P3 profile without converting makes colours look washed out.
4. **Resize** with a good filter (Lanczos) to each target width, keeping the aspect ratio; crop only where a fixed box is needed (avatars).
5. **Encode** each format at a tuned quality, and **strip metadata**: GPS location and camera serials are a privacy leak, and EXIF can be tens of KB.

Use **libvips** (sharp, pyvips, imgproxy): it streams through the image and is typically much faster and lighter on memory than ImageMagick.

3 Formats matter as much as size

Format	Compared with JPEG	Browser support	Notes
JPEG	baseline	universal	fallback; use progressive / mozjpeg encoding
WebP	typically 25–35% smaller at similar quality	all modern browsers (Safari 14+)	fast to encode; supports alpha and animation
AVIF	often around 50% smaller	Chrome, Firefox, Safari 16+	slow and CPU-heavy to encode; great for large hero images
PNG	much larger for photos	universal	only for graphics needing lossless or sharp edges

Serve the best format each browser supports. Either the HTML chooses (`<picture>` with `<source type>`), or the server negotiates on the request's `Accept` header (`image/avif,image/webp,…`) and replies with `Vary: Accept`. With negotiation, make the CDN cache key include the *normalised* format (avif / webp / jpeg), not the raw Accept string, or the hit rate collapses (lesson 11).

```
<picture>
  <source type="image/avif" srcset="/t/42/600.avif 600w, /t/42/1200.avif 1200w">
  <source type="image/webp" srcset="/t/42/600.webp 600w, /t/42/1200.webp 1200w">
  <img src="/t/42/600.jpg" srcset="/t/42/150.jpg 150w, /t/42/600.jpg 600w,
       /t/42/1200.jpg 1200w" sizes="(max-width: 600px) 100vw, 600px"
       width="600" height="450" loading="lazy" alt="…">
</picture>
```

Figure 2. Responsive images: the browser picks the first format it supports and, via `srcset` / `sizes`, the smallest width that fills the slot at its pixel ratio (600w for a 300 px slot on a 2× screen). Explicit width/height prevent layout shift.

4 On-demand transformation

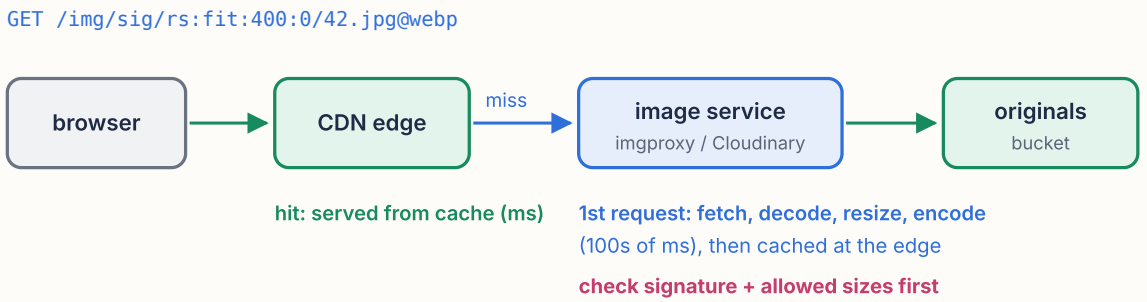


Figure 3. On demand: the URL describes the variant; the image service makes it on the first request and the CDN caches it.

The URL itself describes the variant, e.g. `/img/<sig>/rs:fit:400:0/42.jpg@webp` (imgproxy) or `/w_400,f_auto/42.jpg` (Cloudinary-style). On a CDN miss, an image service fetches the original, transforms it and returns it with long cache headers; every later request is an edge hit. Options: self-hosted **imgproxy** or **thumbor**; managed **Cloudinary**, **imgix**, **Cloudflare Images**; or AWS's Dynamic Image Transformation for CloudFront solution (Lambda + sharp).

	Pre-generated	On demand
Latency	every request is a cache or storage hit	first request per variant per region is slow (100s of ms)
Storage	all variants for all images, used or not	only variants actually requested (in cache)
New size or format	backfill job over every original	change the URL; nothing to reprocess
Risk	simple, predictable cost	abuse: arbitrary sizes force endless transforms
Best when	few, known sizes (avatars, product grids)	many sizes, formats, art direction, frequent redesigns

A backfill is not trivial: adding one size for 50 million images at ~100 ms each is 5 million CPU-seconds, about 58 CPU-days, plus the new storage. That's the usual reason teams move to on-demand transforms.

5 Caching and guard rails

- **Cache forever, change the URL**: variants get `Cache-Control: public, max-age=31536000, immutable`, and the URL includes a version or content hash, so an edited image gets a new URL instead of a stale cache (no purges needed, lesson 11).
- **Allowlist variants**: accept only known widths (e.g. 150, 300, 600, 1200; round requests up) and formats, or **sign** transform URLs (imgproxy uses an HMAC key and salt) so only your app can mint them. Otherwise anyone can request `w=1..5000` and burn CPU and cache.
- **Decompression bombs**: a 50 KB PNG can declare 50,000 × 50,000 pixels, which is 10 GB of RAM at 4 bytes per pixel. Check dimensions from the header and cap pixels (Pillow refuses images over about 179 megapixels by default; set your own lower limit) and input bytes.
- **Isolate decoders**: image parsers are a classic attack surface (ImageTragick, CVE-2016-3714, gave remote code execution via ImageMagick). Run them in sandboxed, low-privilege workers with timeouts and memory limits, and keep libraries patched.
- **Origin shield** in front of the image service collapses simultaneous first requests for the same variant into one transform.

TRADE-OFFS

- Pre-generate: fast and predictable, but storage for unused variants and costly backfills.
- On demand: flexible and lean, but slow first hits, more moving parts and an abuse surface.
- AVIF: smallest files, but slow encoding; often worth it only for large or popular images.
- Higher compression saves bytes but can show artefacts; tune quality per format with visual metrics (SSIM, Butteraugli).

6 Example: three thumbnail sizes after upload

WORKED EXAMPLE: 150, 600 AND 1200 PX IN WEBP AND JPEG

Photo 42 lands as `uploads/42.jpg` (8 MB) and an event goes onto a queue. A worker downloads it, checks its 12 MP size against a 50 MP cap, auto-orients, converts to sRGB, and produces widths **150, 600 and 1200** in **WebP and JPEG**: six objects under predictable keys such as `thumbs/42/600.webp`, together around 0.3–0.5 MB instead of 8 MB per view. It writes the variant keys to the photo's row and marks it `ready`; the page uses a `<picture>` element pointing at the CDN, which caches each variant for a year. If the worker crashes, the queue redelivers and the job simply overwrites the same keys: it's idempotent.

COMMON MISTAKES

- Stripping EXIF before applying orientation: sideways photos.
- Leaving GPS metadata in public images.
- Unbounded on-demand sizes, letting anyone trigger expensive transforms.
- Decoding untrusted images in the API process without pixel limits or sandboxing.

7 Key terms

Variant / rendition: resized or re-encoded copy of an original image

srcset / sizes: HTML attributes letting the browser pick the right width

Decompression bomb: tiny file that expands to huge memory when decoded

WebP / AVIF: modern formats, typically 25–50% smaller than JPEG

On-demand transform: variant created on first request from URL parameters, then cached

Auto-orient: apply the EXIF rotation to pixels before stripping metadata

8 Interview questions

Q1. On-demand vs pre-generated image variants? BASIC

Pre-generate when sizes are few and known: simple, and every request is fast. On demand suits many sizes or formats, at the cost of a slower first request and an abuse surface.

Q2. When do on-demand transforms win, and what must you add? BASIC

When you need many sizes or formats or change them often; cache each variant at the CDN, accept a slower first request, and limit or sign the allowed sizes so attackers can't force endless transforms.

Q3. How do you serve WebP or AVIF to browsers that support them and JPEG to others? SCENARIO

Use with typed sources, or negotiate on the Accept header with Vary: Accept and a cache key that includes the normalised format.

Q4. Product wants a new 800 px size for 50 million existing images. Options? SCENARIO

Backfill with a batch job (tens of CPU-days plus storage), generate lazily on first request and store, or switch to an on-demand transform service with CDN caching.

Q5. Your image workers keep running out of memory on some uploads. What's happening? SCENARIO

Likely huge-dimension images or decompression bombs; read dimensions from the header, enforce a pixel cap before decoding, use a streaming library like libvips, and set memory limits.

Q6. Why might thumbnails appear rotated or with dull colours? TRICKY

EXIF orientation was stripped before being applied, and a wide-gamut ICC profile was dropped without converting pixels to sRGB.

Q7. Why put a version or hash in image URLs instead of purging the CDN? TRICKY

A new URL is a guaranteed cache miss everywhere instantly, so you can cache for a year; purges are slow to propagate, rate-limited and miss browser caches.

Next: SD-O-010 · Video Streaming backendroadmap.com